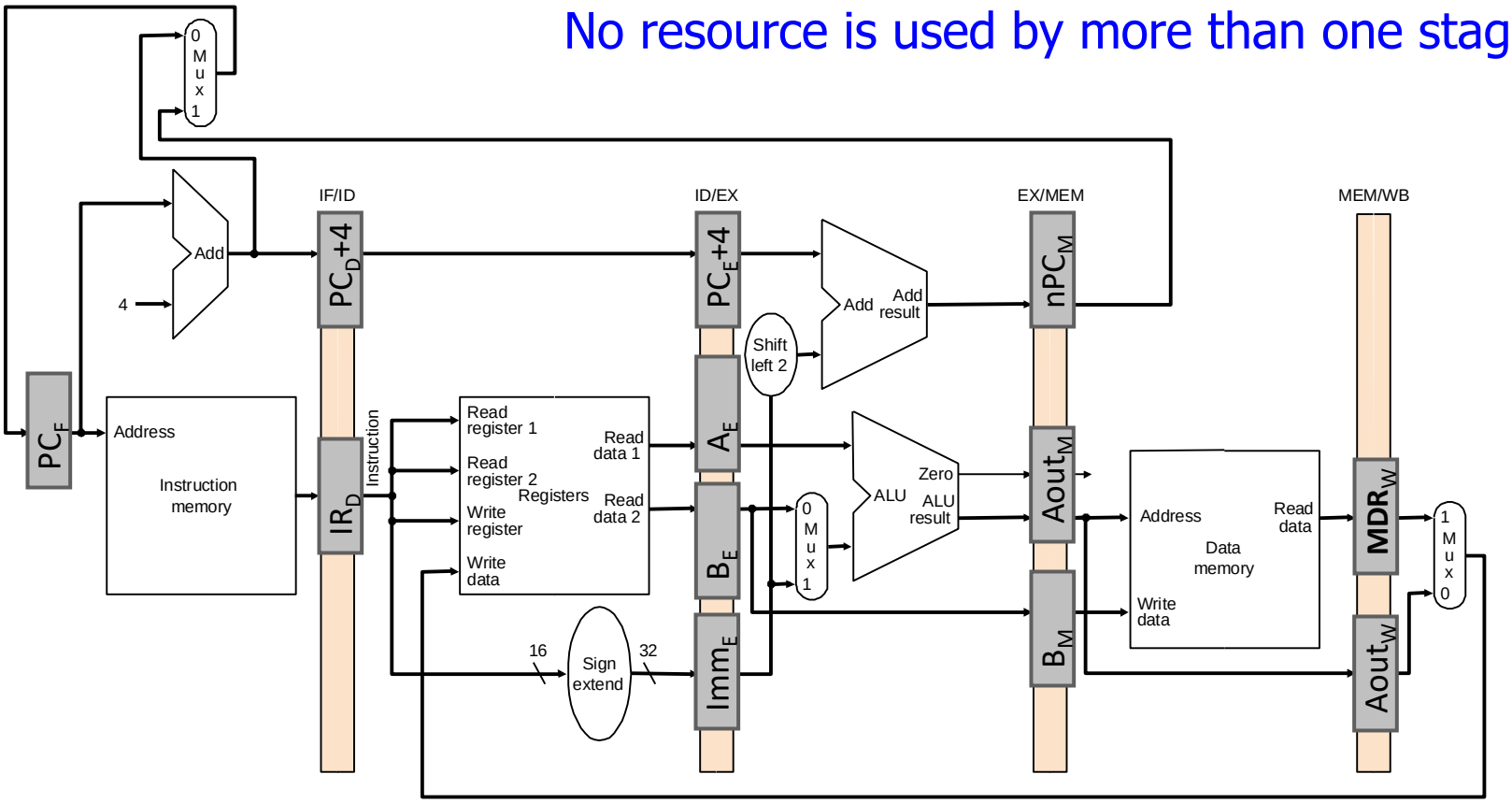
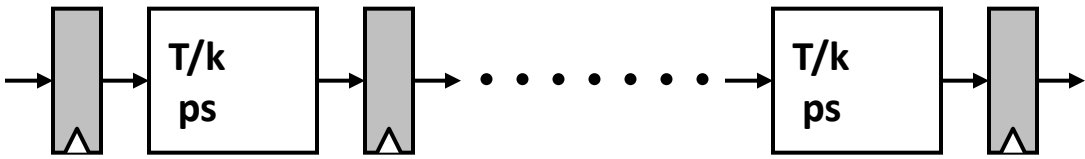
Enabling Pipelined Processing: Pipeline Registers

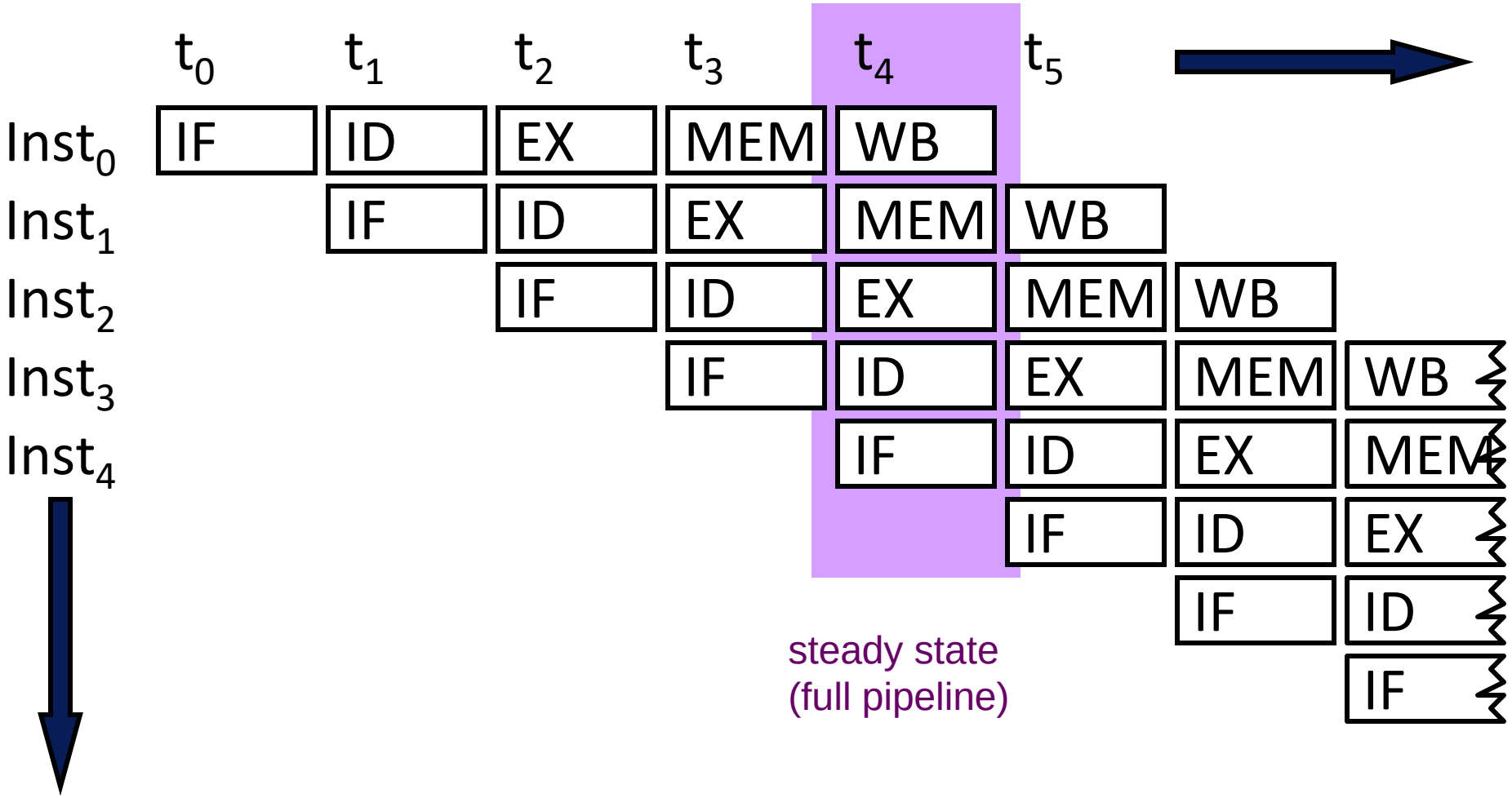
No resource is used by more than one stage





Based on original figure from [P&H CO&D, COPYRIGHT 2004 Elsevier. ALL RIGHTS RESERVED.]

Illustrating Pipeline Operation: Operation View

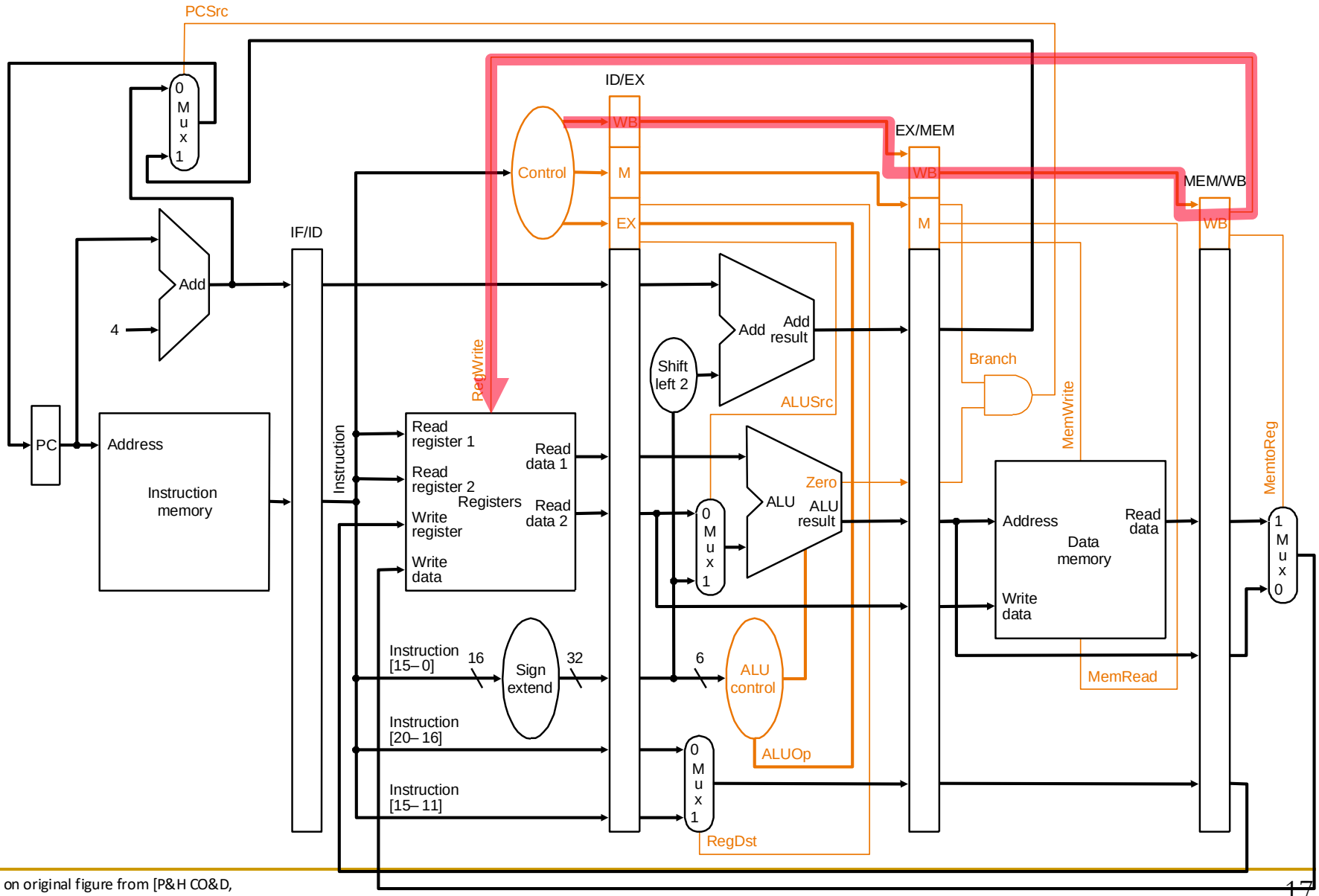


Illustrating Pipeline Operation: Resource View

	t_0	t_1	t_2	t_3	t_4	t_5	t_6	t_7	t_8	t_9	t_{10}
IF	I_0	I_1	I_2	I_3	I_4	I_5	I_6	I_7	I_8	I_9	I_{10}
ID		I_0	I_1	I_2	I_3	I_4	I_5	I_6	I_7	I_8	I_9
EX			I_0	I_1	I_2	I_3	I_4	I_5	I_6	I_7	I_8
MEM				I_0	I_1	I_2	I_3	I_4	I_5	I_6	I_7
WB					I_0	I_1	I_2	I_3	I_4	I_5	I_6

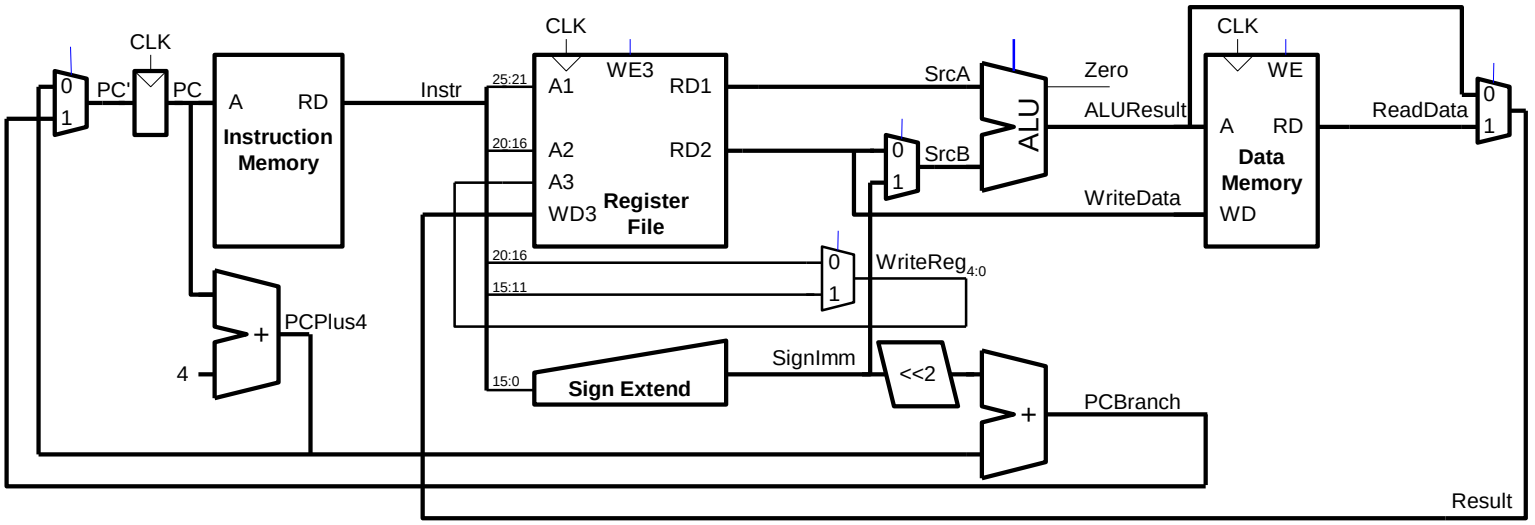
Pipelined Control Signals

Same control unit as single-cycle processor
Control delayed to proper pipeline stage

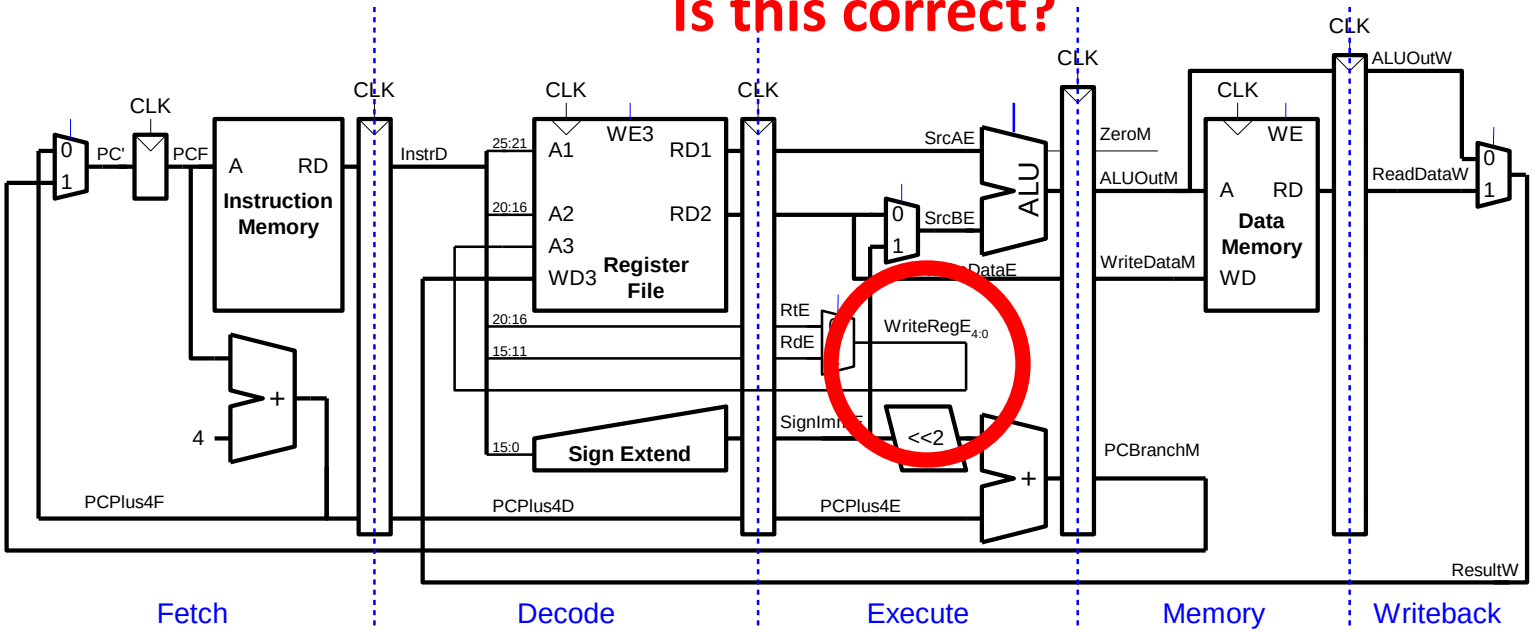


Based on original figure from [P&H CO&D, COPYRIGHT 2004 Elsevier. ALL RIGHTS RESERVED.]

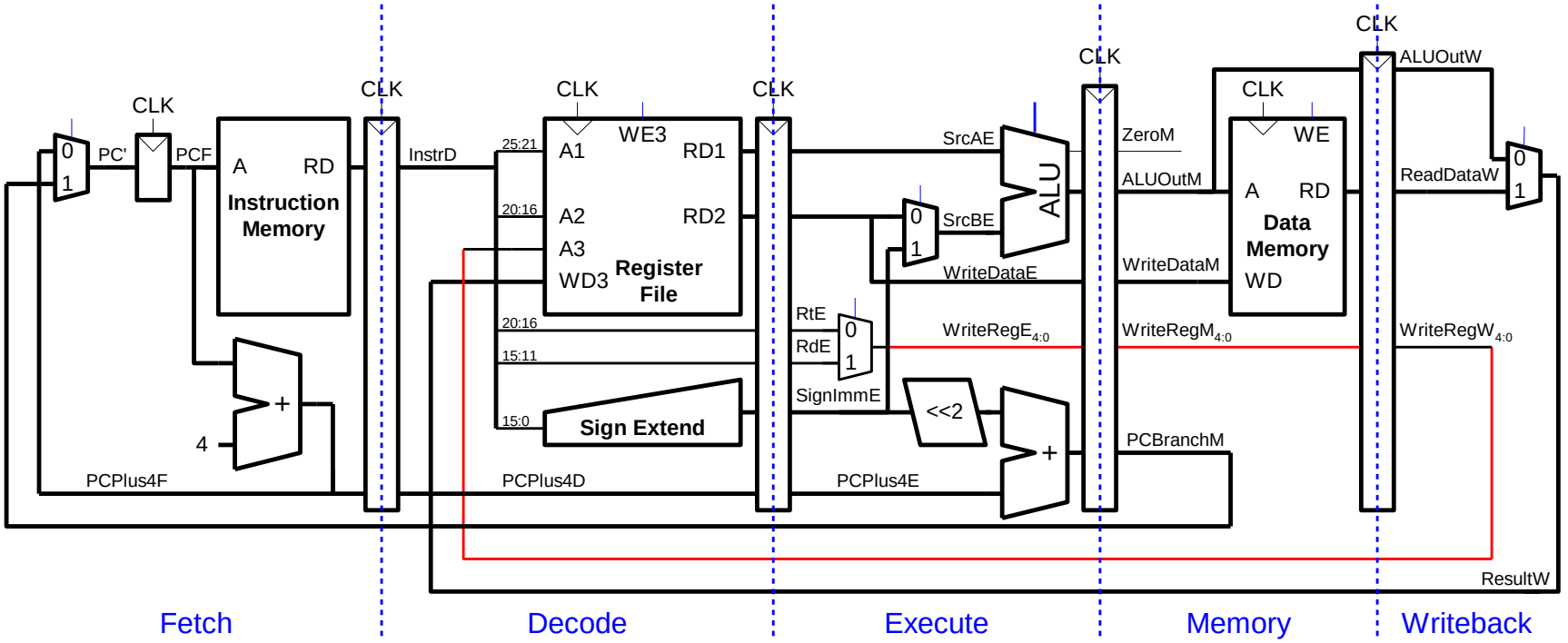
Another Example: Single-Cycle and Pipelined



Is this correct?



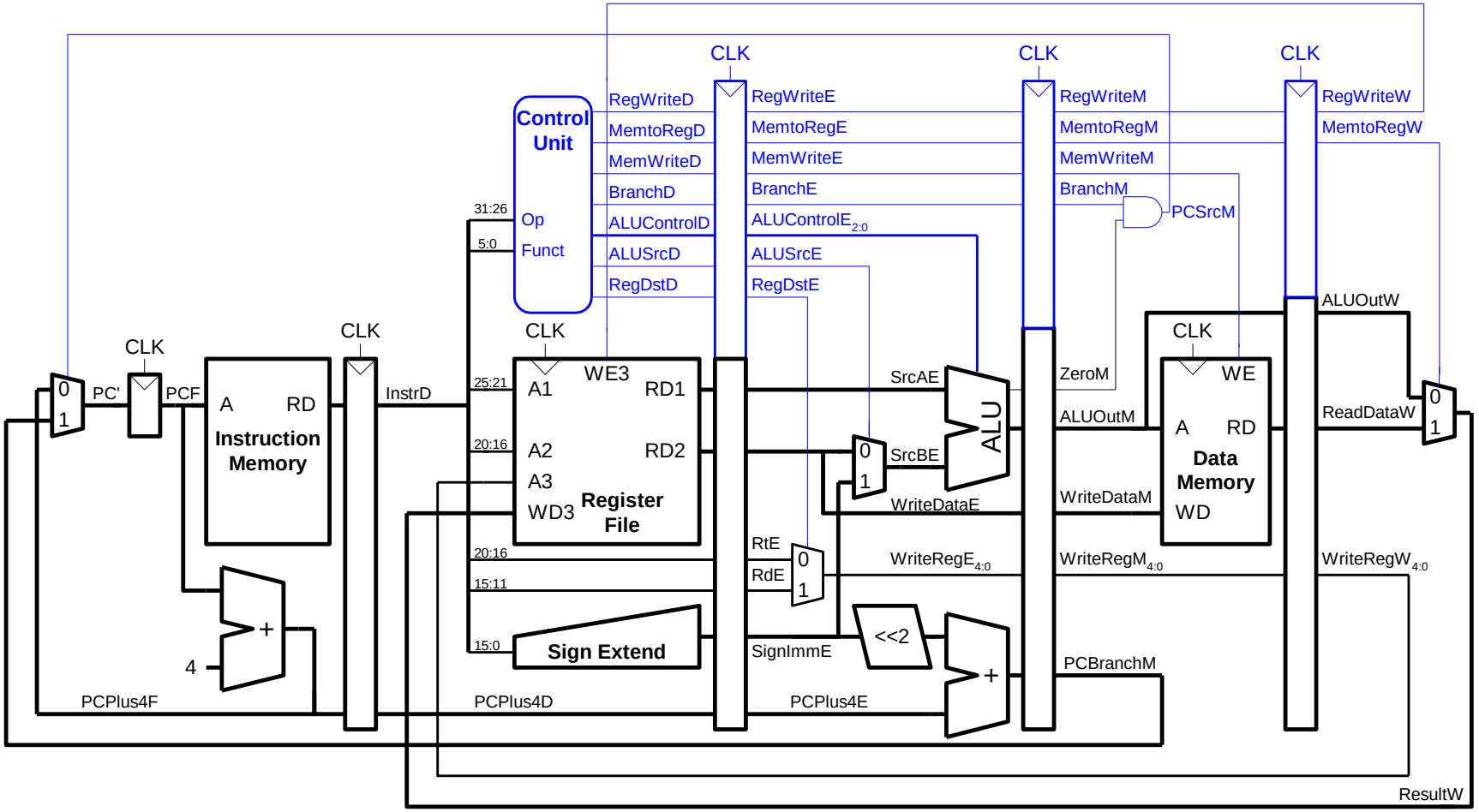
Another Example: Correct Pipelined Datapath



- **WriteReg control signal must arrive at the same time as Result**

Pipelined processor. Harris and Harris, Chapter 7.5

Another Example: Pipelined Control



- **Same control unit as single-cycle processor**
  Control delayed to proper pipeline stage

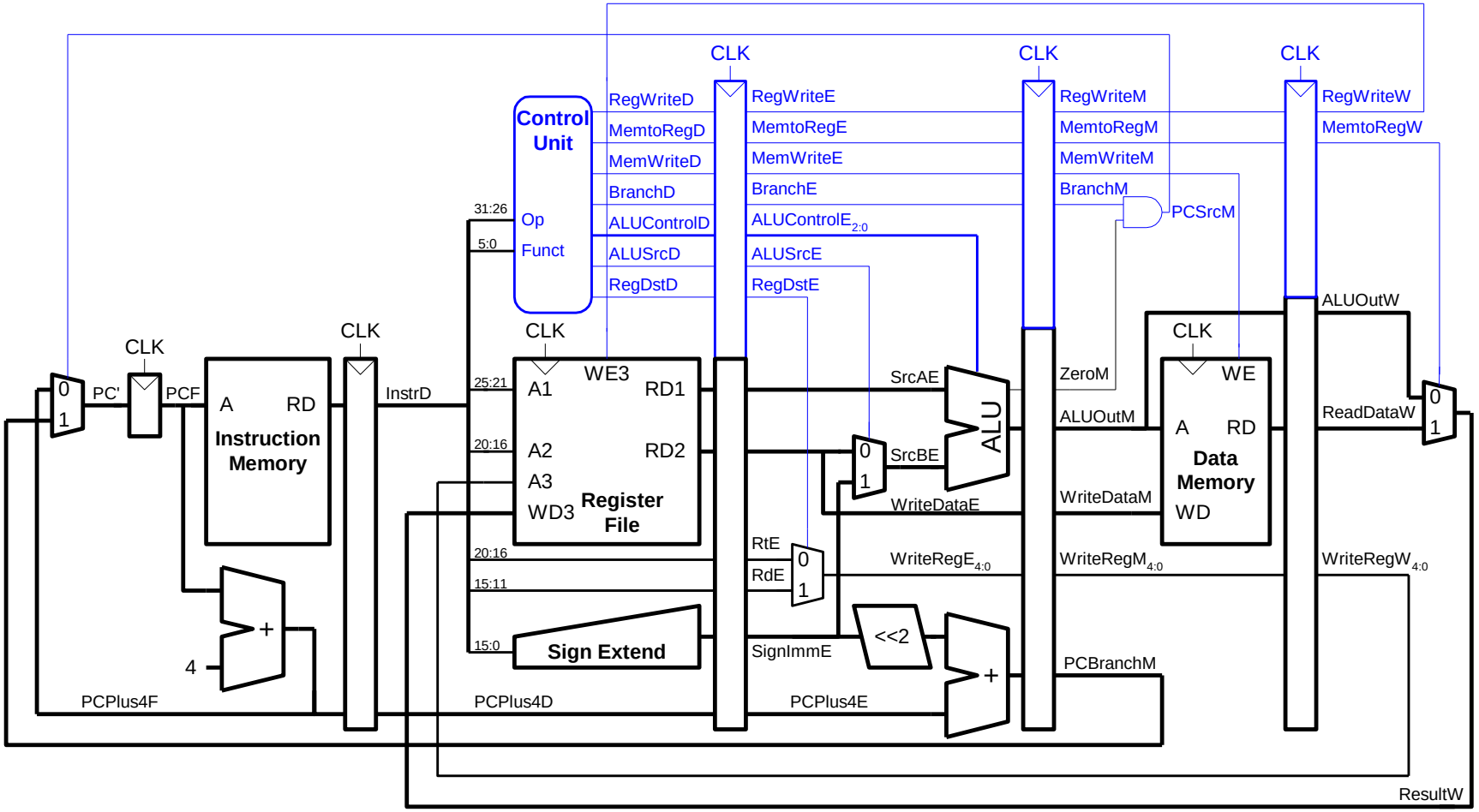
Pipelined Datapath and Control Logic

- Pipelined datapath design
  - Start with the single-cycle datapath
  - Decide how to break it down into stages
  - Add pipeline registers to separate stages from each other
  - Propagate data signals to stages where they are needed
  - Make sure correct action is done in the correct stage
    - Wires that cross stages need to be handled carefully

- Pipelined control logic design
  - Start with the single-cycle control signals and control logic
  - Propagate each control signal to stages where it is needed
    - Need to extend pipeline registers to do so

Is this sufficient?

Basic Pipelined Processor

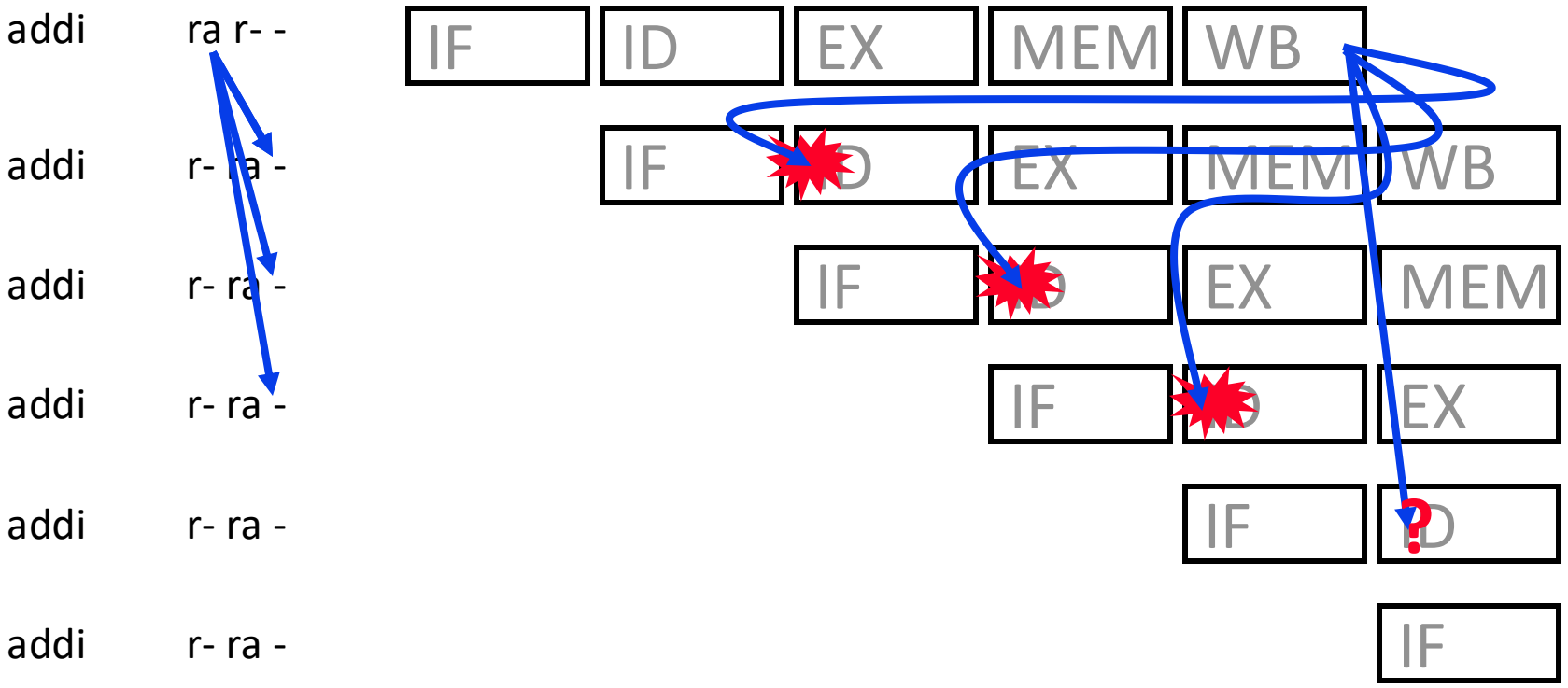


How do we handle dependent instructions?

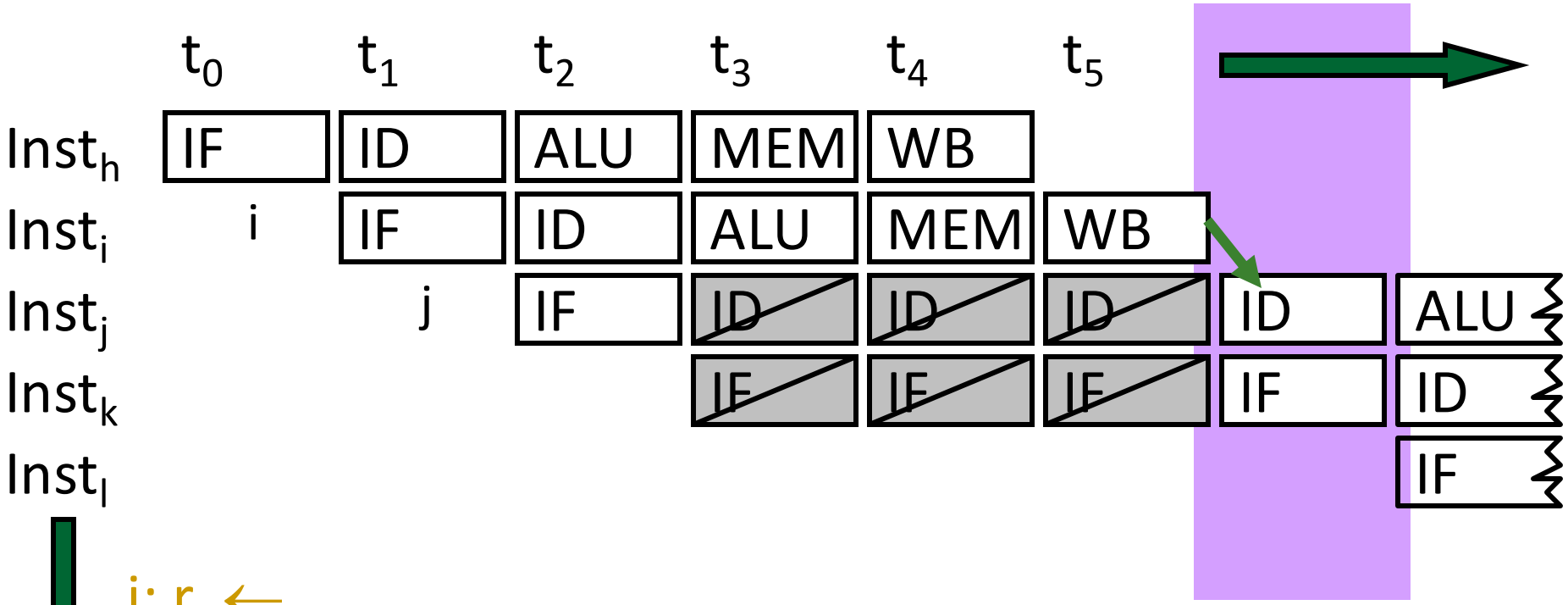
How do we handle multi-cycle memory access?

RAW Dependence Handling

- Which one of the following flow dependences lead to conflicts in the 5-stage pipeline?



Pipeline Stall: Resolving Data Dependence



i: $r_x \leftarrow$ _
bubble
bubble
bubble
j: _ $\leftarrow r_x$

dist(i,j)=4

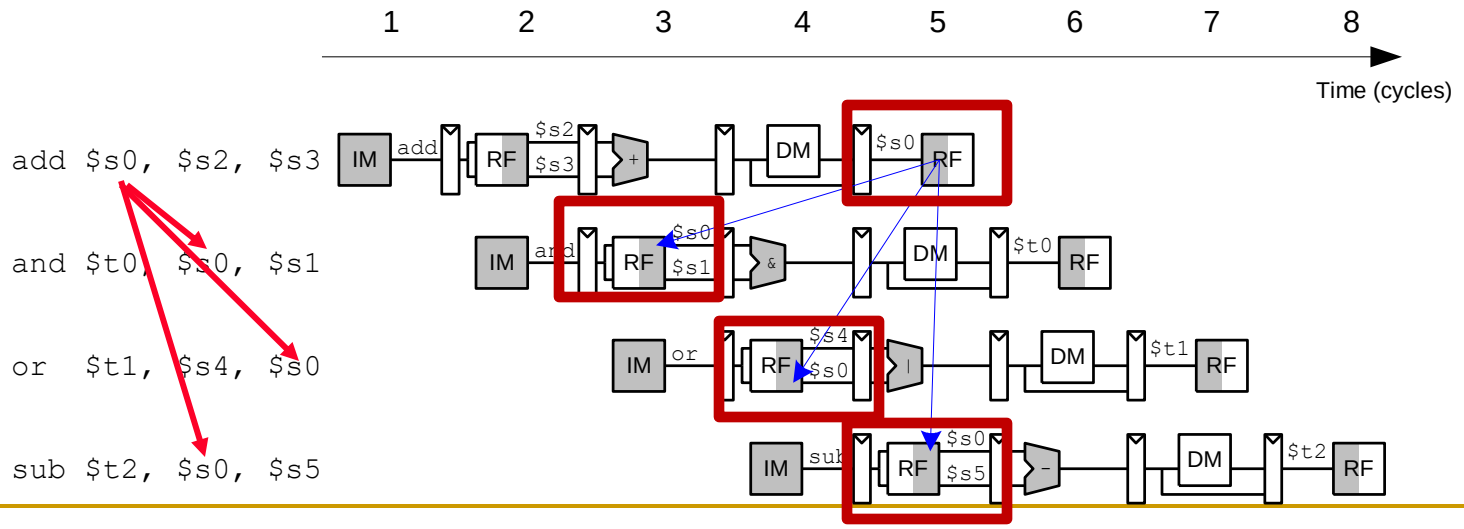
Stall = make the dependent instruction **wait** until its source data value is available
1. stop all up-stream stages
2. drain all down-stream stages

RAW Data Dependence Example

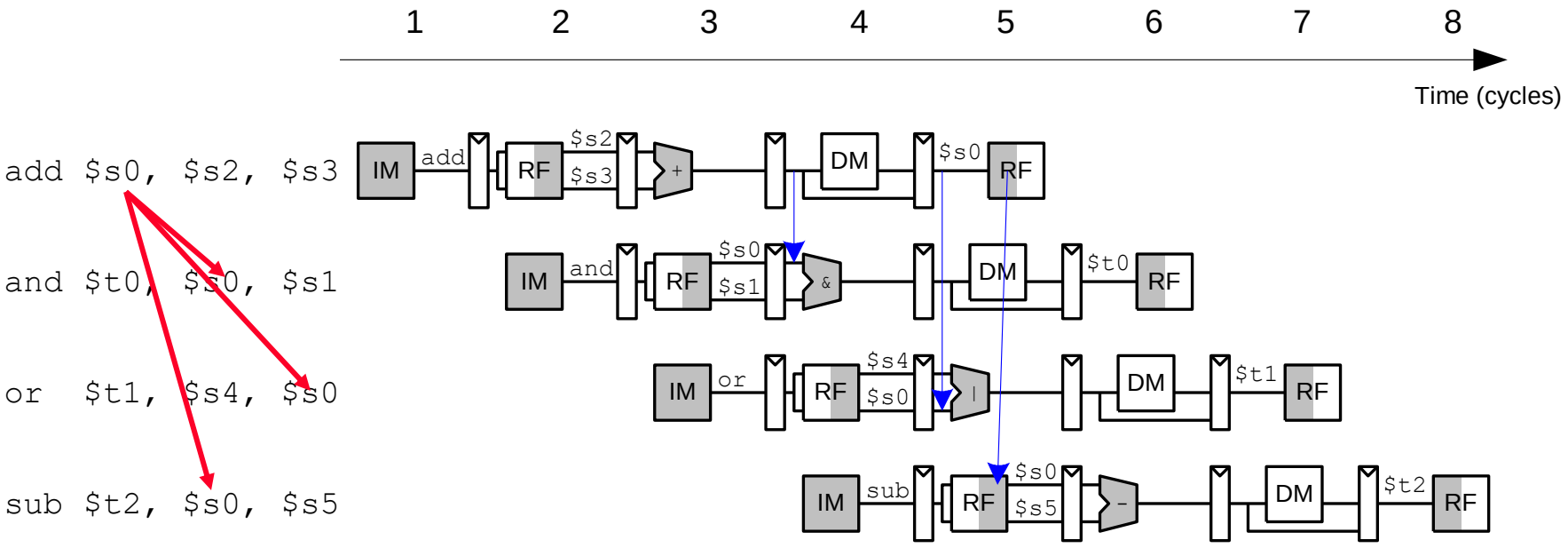
- One instruction writes a register ($s0) and next instructions read this register => read after write (RAW) dependence.

Wrong results happen only if the pipeline handles data dependences incorrectly!

  - subsequent instructions read the correct value of $s0



Data Forwarding: Locations in Datapath



From latched output of ALU to input of ALU

From WB to input of ALU

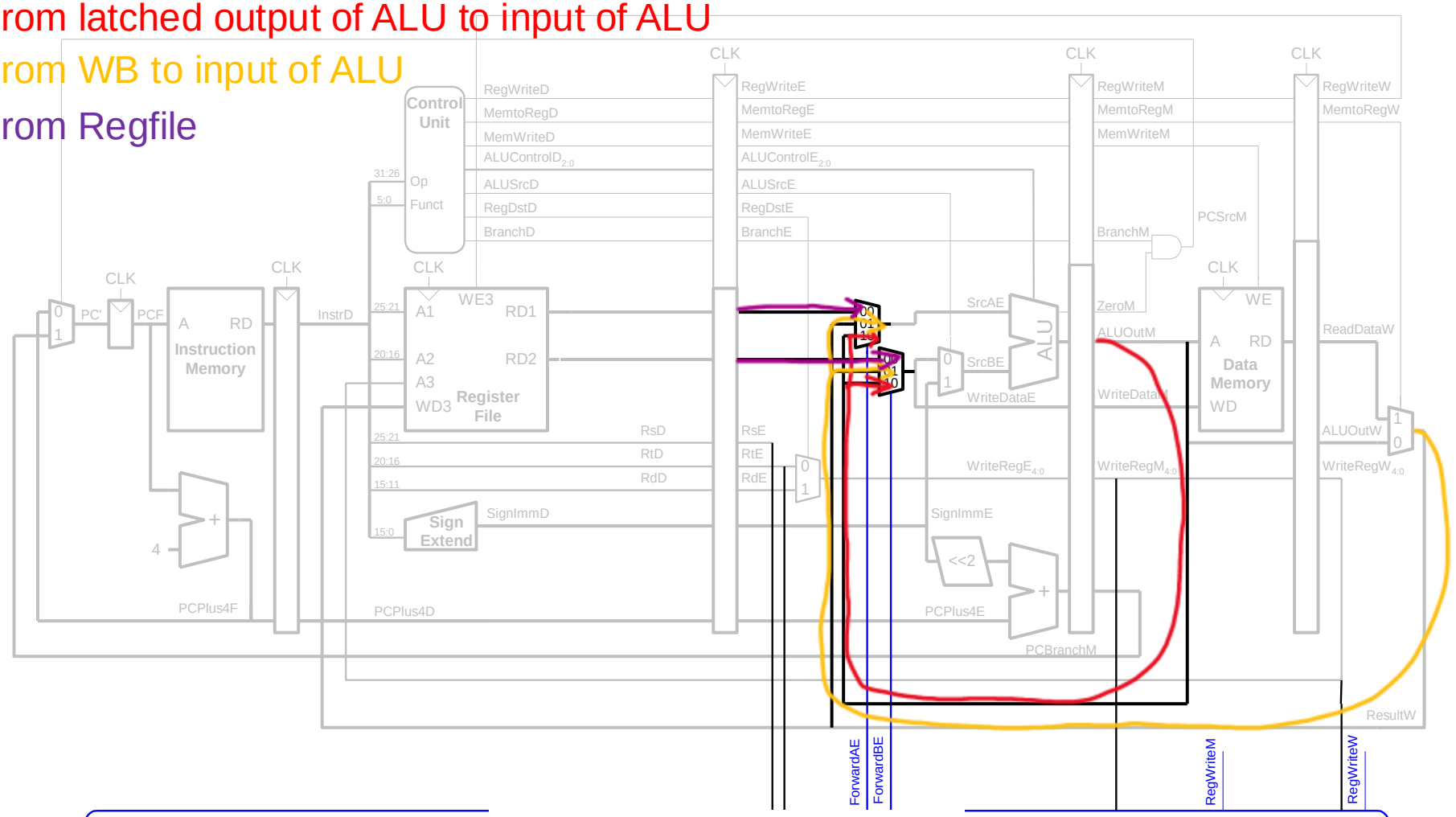
From WB to RF (internal in Register File)

Data Forwarding: Datapath & Control

From latched output of ALU to input of ALU

From WB to input of ALU

From Regfile



Dependence Detection Logic

Data Forwarding: Implementation

- Forward to Execute stage from either:
  - Memory stage or
  - Writeback stage

- When should we forward from either Memory or Writeback stage?
  - If that stage will write to a destination register and the destination register matches the source register
  - If both the Memory & Writeback stages contain matching destination registers, Memory stage has priority to forward its data, because it contains the *more recently executed* instruction

Data Forwarding (in Pseudocode)

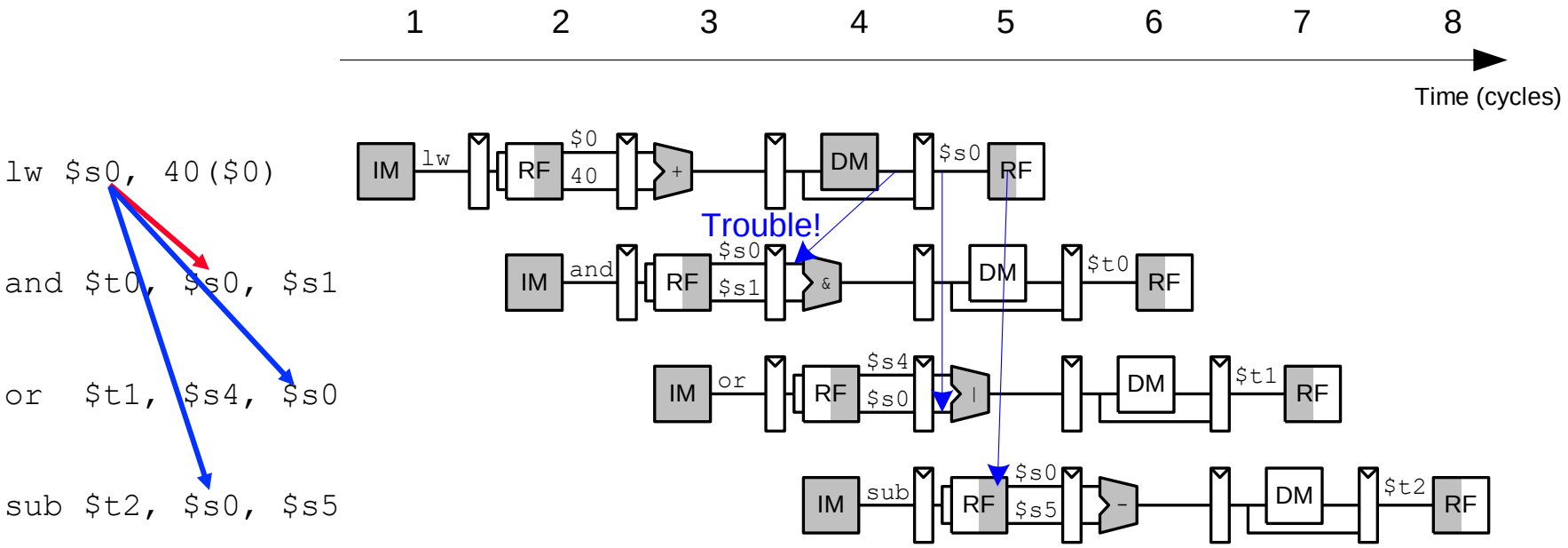
- Forward to Execute stage from either:
  - Memory stage or
  - Writeback stage

- Forwarding logic for *ForwardAE (pseudo code)*:

```
if  ((rsE != 0) AND (rsE == WriteRegM) AND RegWriteM) then
      ForwardAE = 10  # forward from Memory stage
else if ((rsE != 0) AND (rsE == WriteRegW) AND RegWriteW) then
      ForwardAE = 01  # forward from Writeback stage
else
      ForwardAE = 00  # no forwarding
```

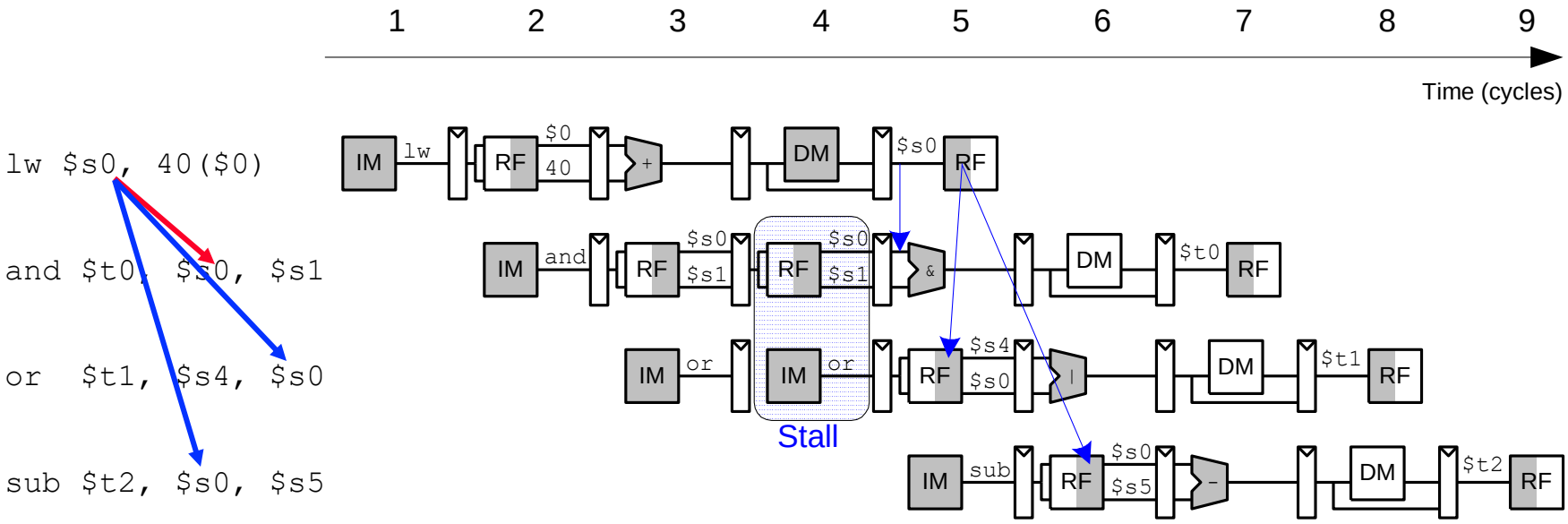
- Forwarding logic for *ForwardBE* same, but replace *rsE* with *rtE*

Data Forwarding Is **Not** Always Possible

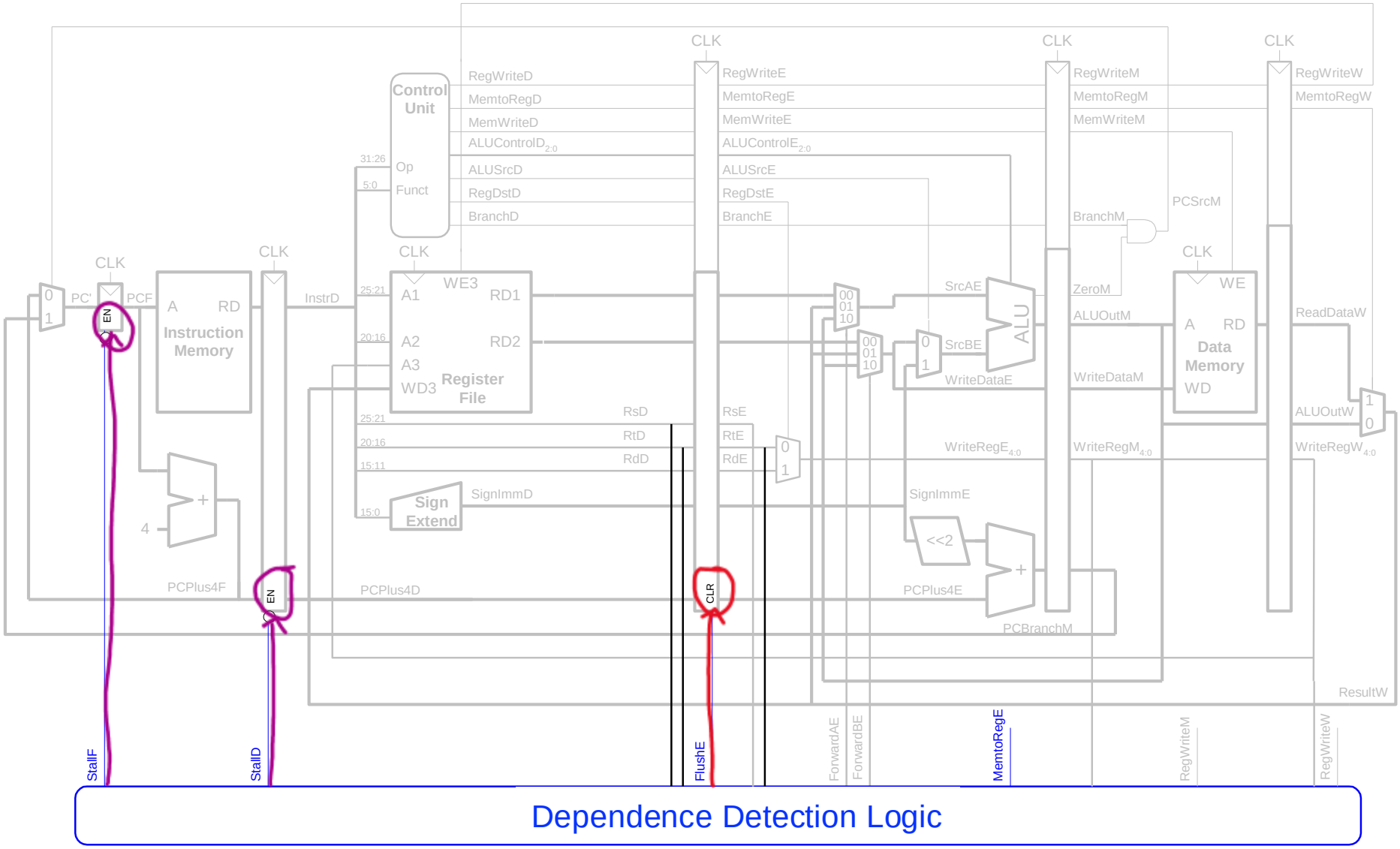


- Forwarding is usually sufficient to resolve RAW data dependences
- Unfortunately, there are cases when forwarding is **not possible**
  - due to pipeline design and instruction latencies
  - The `lw` instruction does not finish reading data until the end of Memory stage
    → its result cannot be forwarded to the Execute stage of the next instruction
    unless we want a long critical path → breaks critical path design principle

Stalling Necessary for MEM-EX Dependence



Stalling and Dependence Detection Hardware



Dependence Detection Logic

Control Dependence

- Question: What should the fetch PC be in the next cycle?
- Answer: The address of the next instruction
  - All instructions are control dependent on previous ones. Why?

- If the fetched instruction is a non-control-flow instruction:
  - Next Fetch PC is the address of the next-sequential instruction
  - Easy to determine if we know the size of the fetched instruction

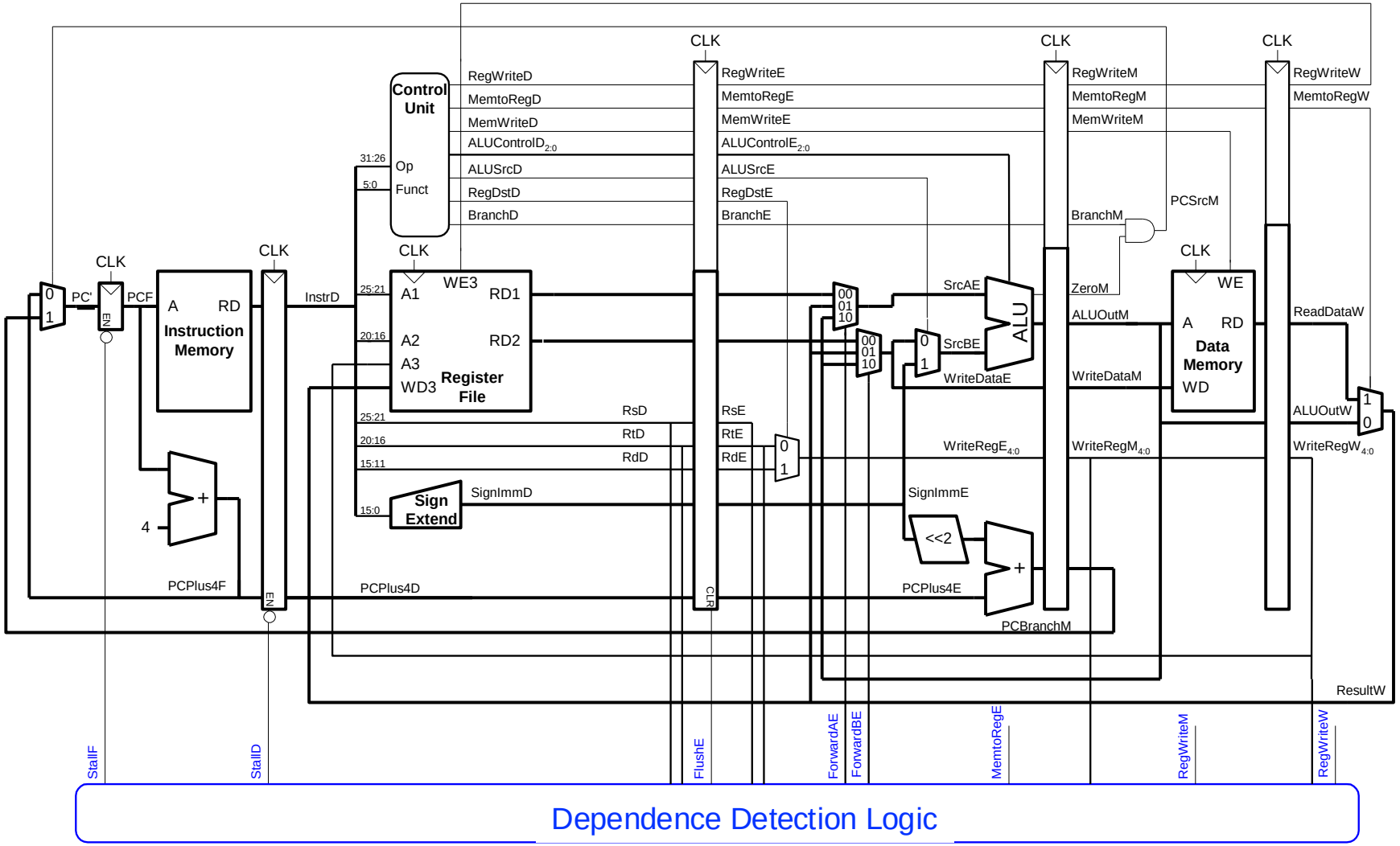
- If the instruction that is fetched is a control-flow instruction:
  - How do we determine the next Fetch PC?

- In fact, how do we know whether or not the fetched instruction is a control-flow instruction?

Branch Prediction

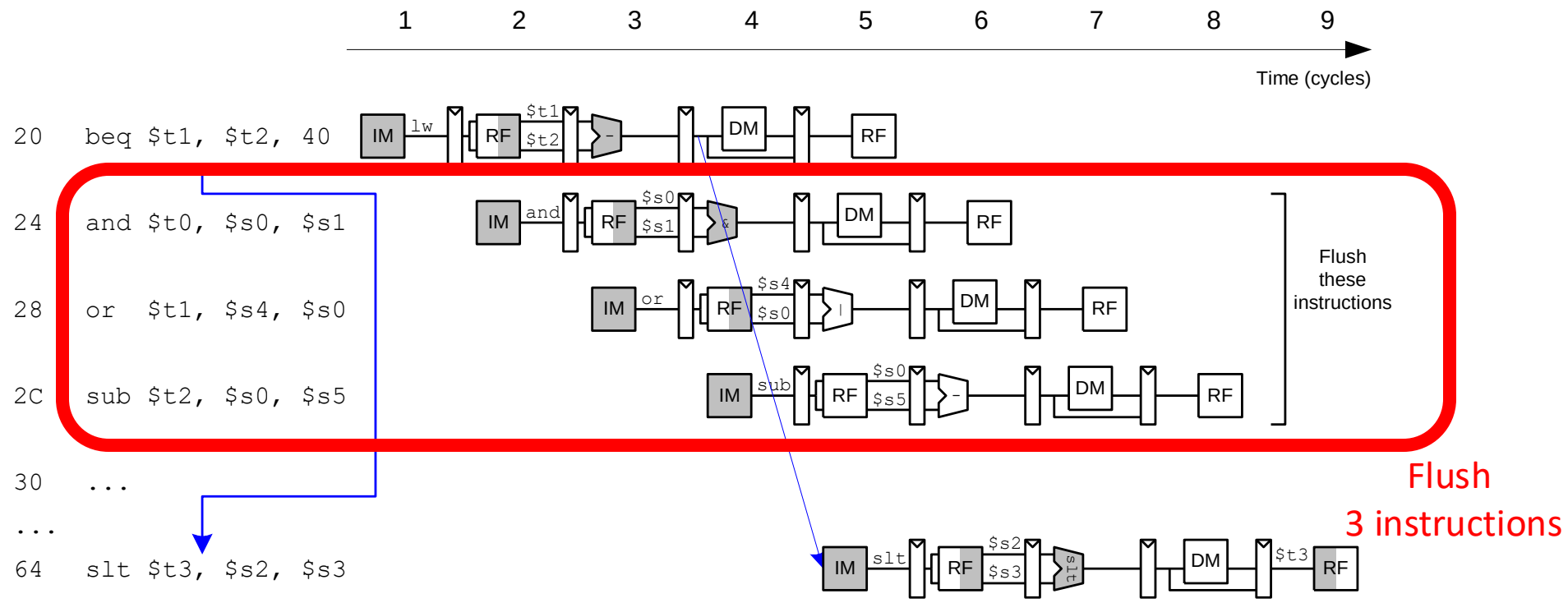
- **Special case of data dependence: dependence on PC**
- **beq:**
  - Conditional branch is not resolved until the fourth stage of the pipeline
  - **Instructions after the branch are fetched before branch is resolved**
  - Simple "**branch prediction**" example:
    - Always predict that the next sequential instruction is fetched
    - Called "**Always not taken**" prediction
    - Flush (invalidate) "not-taken path" instructions if the branch is taken
- **Branch misprediction penalty**
  - **number of instructions flushed when branch is incorrectly predicted**
  - Penalty can be reduced by resolving the branch earlier
    - Called "**Early branch resolution**"

Our Pipeline So Far



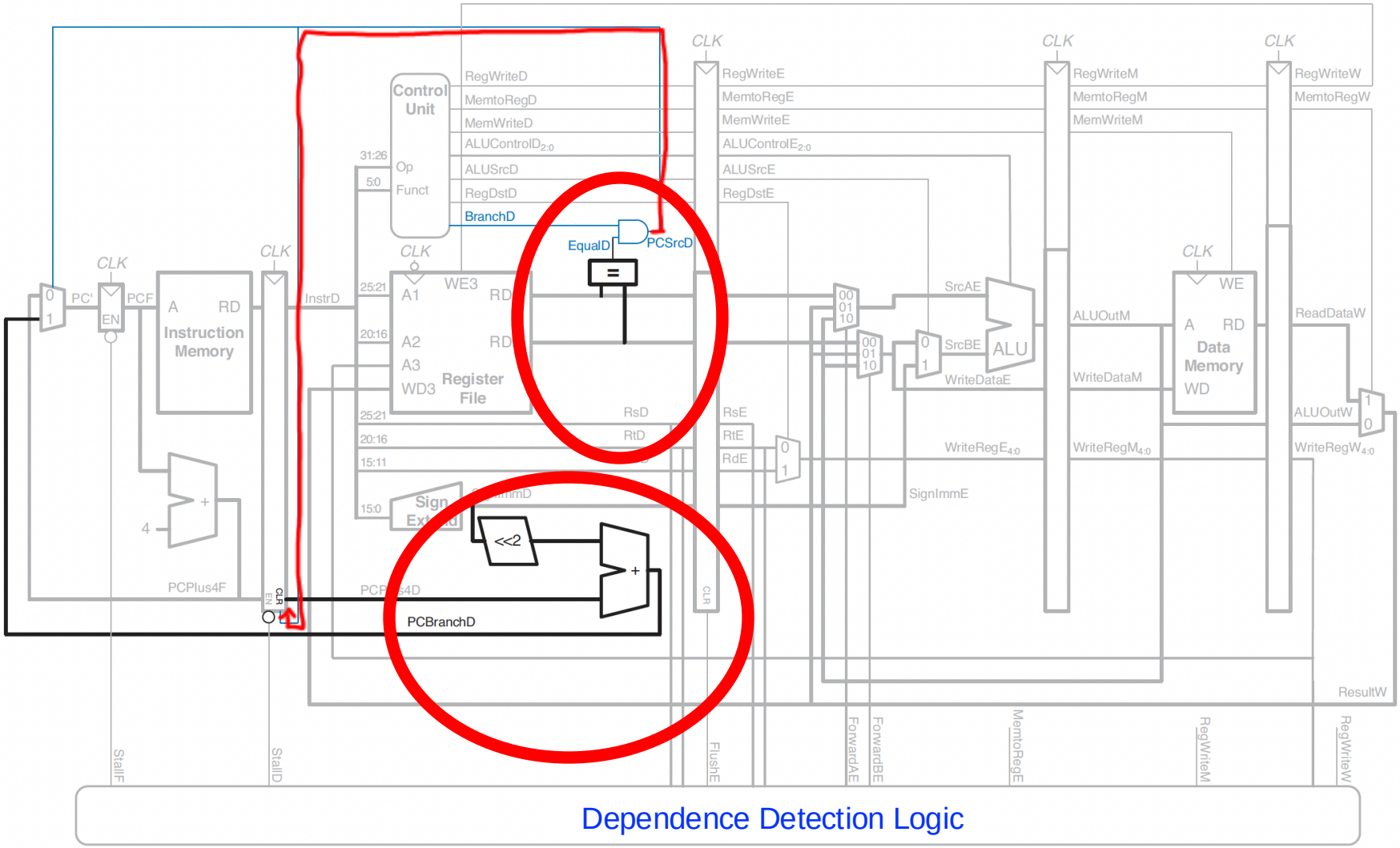
Dependence Detection Logic

Control Dependence: Flush on Misprediction



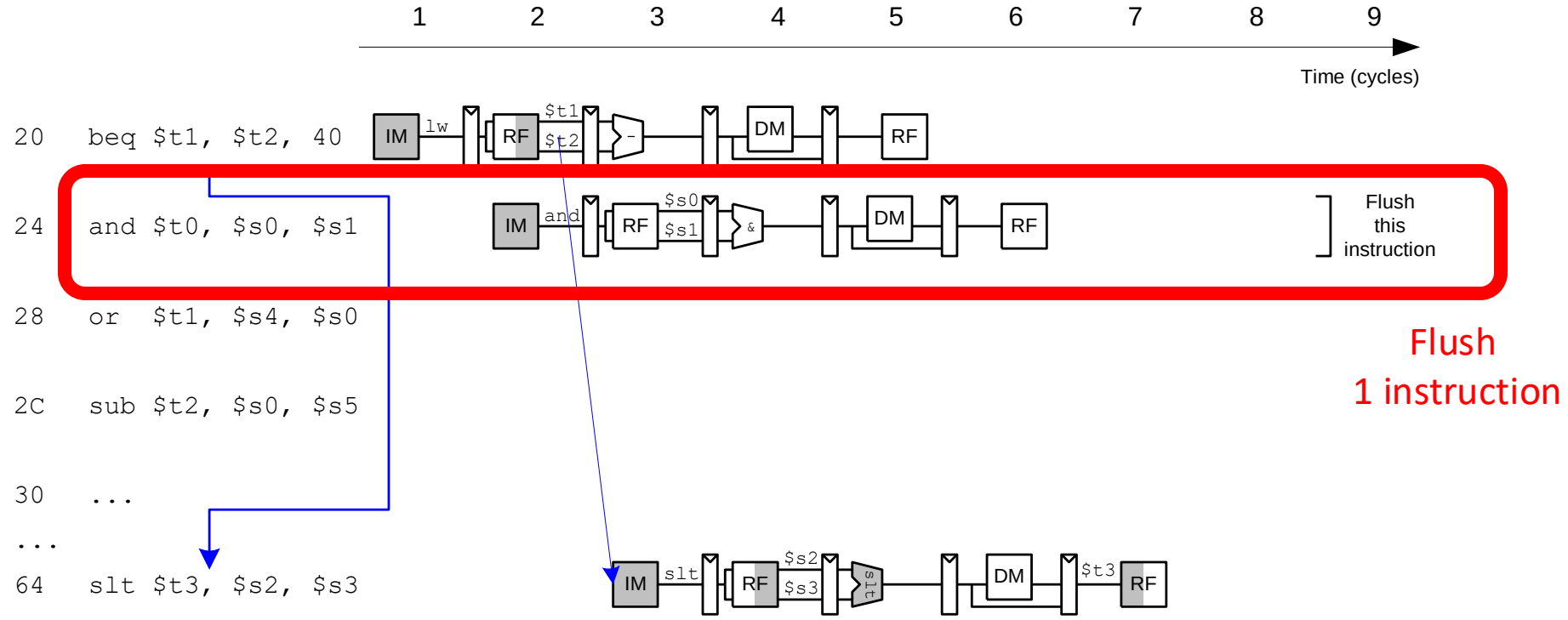
Flush
3 instructions

Pipeline with Early Branch Resolution



Need to calculate branch target and condition in the Decode Stage

Early Branch Resolution



Flush
1 instruction

Forwarding and Stalling Hardware Control

```
// Forwarding logic:
assign ForwardAD = (rsD != 0) & (rsD == WriteRegM) & RegWriteM;
assign ForwardBD = (rtD != 0) & (rtD == WriteRegM) & RegWriteM;

//Stalling logic:
assign lwstall = ((rsD == rtE) | (rtD == rtE)) & MemtoRegE;

assign branchstall = (BranchD & RegWriteE &
                      (WriteRegE == rsD | WriteRegE == rtD))
                      |
                      (BranchD & MemtoRegM &
                      (WriteRegM == rsD | WriteRegM == rtD));

// Stall signals;
assign StallF = lwstall | branchstall;
assign StallD = lwstall | branchstall;
assign FLushE = lwstall | branchstall;
```

Final Pipelined MIPS Processor (H&H)

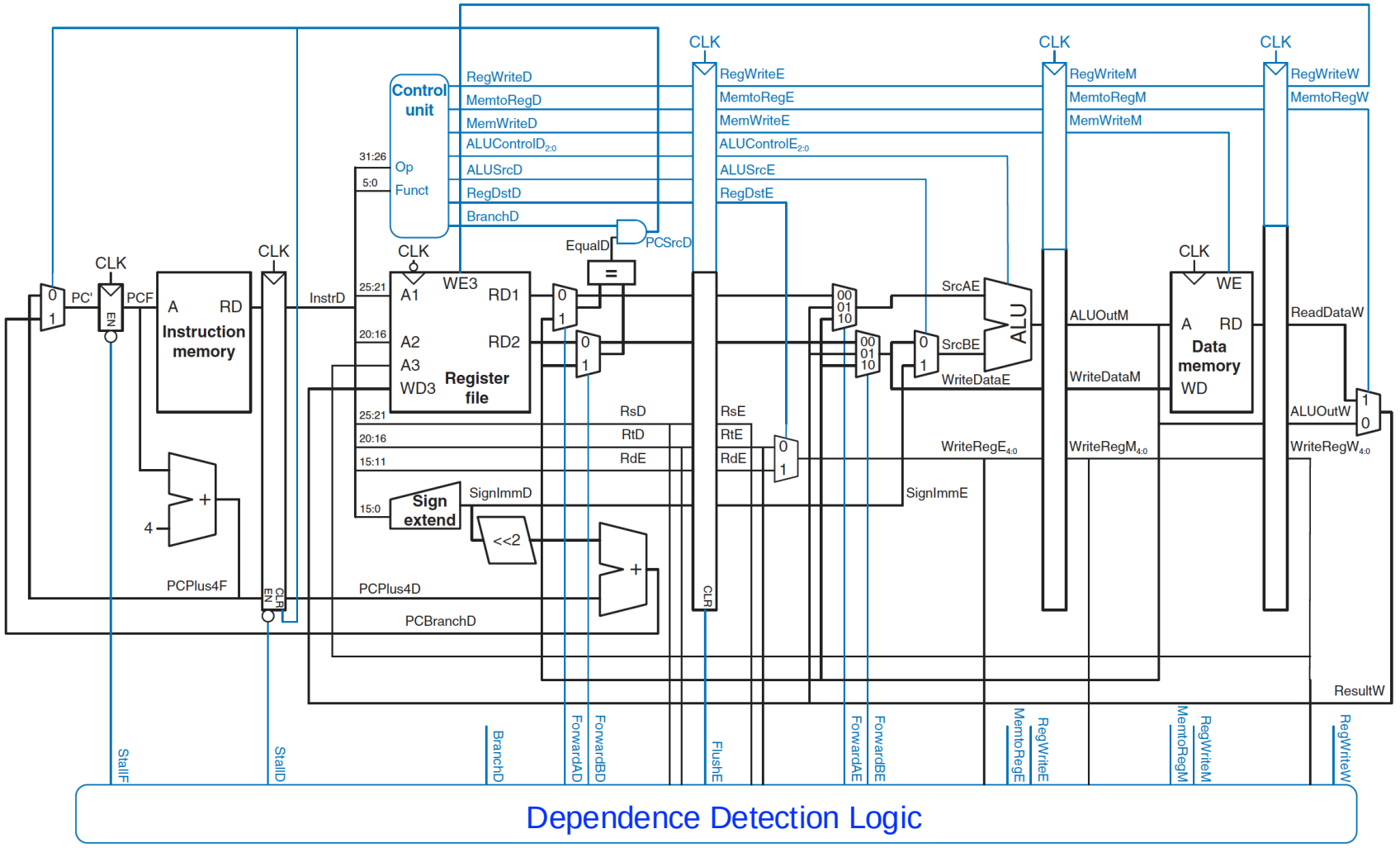


Figure 7.58 Pipelined processor with full hazard handling

Includes always-not-taken br prediction, early branch resolution, forwarding, stall logic

Pipelined Performance Example

Pipelined Performance Example

- **An important program consists of:**
  - 25% loads
  - 10% stores
  - 11% branches
  - 2% jumps
  - 52% R-type
- **Assume:**
  - 40% of loads used by next instruction
  - 25% of branches mispredicted; flushes the next instruction on misprediction
  - All jumps flush the next instruction fetched
- **What is the average CPI?**

Pipelined Performance Example: CPI

- **Load/Branch CPI = 1 when no stall/flush, 2 when stall/flush. Thus:**
  - $CPI_{lw} = 1(0.6) + 2(0.4) = 1.4$ *Average CPI for load*
  - $CPI_{beq} = 1(0.75) + 2(0.25) = 1.25$ *Average CPI for branch*
- **And**
  - ***Average CPI*** =

Pipelined Performance Example: CPI

- **Load/Branch CPI = 1 when no stall/flush, 2 when stall/flush. Thus:**
  - $CPI_{lw} = 1(0.6) + 2(0.4) = 1.4$ — *Average CPI for load*
  - $CPI_{beq} = 1(0.75) + 2(0.25) = 1.25$ — *Average CPI for branch*
- **And**
  - ***Average CPI*** = (0.25)(1.4) + *load*
    (0.1)(1) + *store*
    (0.11)(1.25) + *beq*
    (0.02)(2) + *jump*
    (0.52)(1) *r-type*

    = **1.15**

Pipelined Performance Example: Cycle Time

- There are 5 stages, and 5 different timing paths:

T_c = max {

$t_{pcq} + t_{mem} + t_{setup}$	*fetch*
$2(t_{RFread} + t_{mux} + t_{eq} + t_{AND} + t_{mux} + t_{setup})$	*decode*
$t_{pcq} + t_{mux} + t_{mux} + t_{ALU} + t_{setup}$	*execute*
$t_{pcq} + t_{memwrite} + t_{setup}$	*memory*
$2(t_{pcq} + t_{mux} + t_{RFwrite})$	*writeback*

}

- The clock cycle ***depends*** on the ***slowest stage***

- Decode and Writeback use register file and have only half a clock cycle to complete → that is why there is a 2 in front of them

Final Pipelined MIPS Processor (H&H)

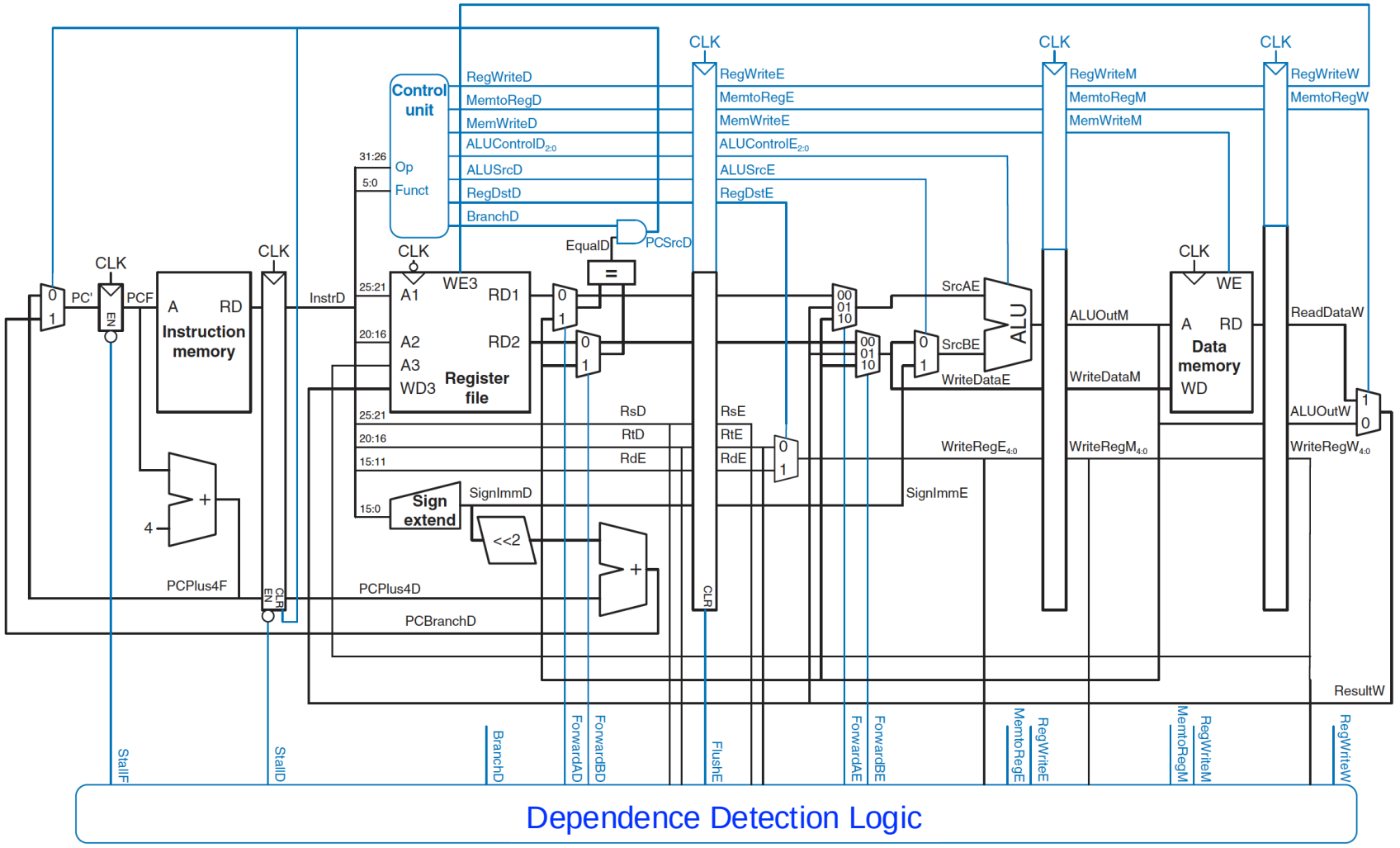


Figure 7.58 Pipelined processor with full hazard handling

Includes always-not-taken br prediction, early branch resolution, forwarding, stall logic

Pipelined Performance Example: Cycle Time

Element	Parameter	Delay (ps)
Register clock-to-Q	t_{pcq_PC}	30
Register setup	t_{setup}	20
Multiplexer	t_{mux}	25
ALU	t_{ALU}	200
Memory read	t_{mem}	250
Register file read	t_{RFread}	150
Register file setup	$t_{RFsetup}$	20
Equality comparator	t_{eq}	40
AND gate	t_{AND}	15
Memory write	$T_{memwrite}$	220
Register file write	$t_{RFwrite}$	100

$$T_c = 2(t_{RFread} + t_{mux} + t_{eq} + t_{AND} + t_{mux} + t_{setup})$$
$$= 2[150 + 25 + 40 + 15 + 25 + 20] \text{ ps}$$
$$= 550 \text{ ps}$$

Pipelined Performance Example: Exec Time

- **For a program with 100 billion instructions executing on a pipelined MIPS processor:**
  - CPI = 1.15
  - T_c = 550 ps

- **Execution Time** **= (# instructions) × CPI × T_c**
  = $(100 \times 10^9)(1.15)(550 \times 10^{-12})$
  = 63 seconds

Performance Summary for 3 MIPS microarch.

Processor	Execution Time (seconds)	Normalized Execution Time (single-cycle is baseline)
Single-cycle	95	1
Multicycle	133	0.71
Pipelined	63	1.51

- **Pipelined implementation is the fastest of 3 implementations**
- **Even though we have a 5-stage pipeline, speedup is not 5X over the single-cycle or the multi-cycle system!**